

# **AI Medical Assessment System**

## **Introduction**

The AI Medical Assistant is a simple healthcare support application developed using Python and Streamlit. The primary purpose of this system is to provide users with basic medical guidance based on the symptoms they enter. The application analyzes user input and offers general health information, self-care recommendations, and suggestions on when professional medical attention may be necessary.

With the increasing demand for quick access to health-related information, AI-powered healthcare assistants can help users better understand common symptoms and make informed decisions about their well-being. This project demonstrates how artificial intelligence concepts and web technologies can be combined to create an interactive and user-friendly medical assistance platform.

The system is designed for educational and informational purposes only. It does not provide medical diagnoses and should not be considered a substitute for consultation with qualified healthcare professionals. The application aims to improve awareness about common health conditions while encouraging users to seek professional medical advice when needed.

## **Project Description**

The AI Medical Assistant is a web-based application developed using Python and Streamlit that provides basic health-related guidance based on user-entered symptoms. Users can describe their symptoms or ask general health questions, and the system analyzes the input to identify common conditions such as fever, cold, and headache.

Based on the detected symptoms, the application provides general information, self-care recommendations, and advice on when medical attention may be required. The system is designed with a simple and interactive user interface, making it easy for users to access health information quickly.

The application serves as an educational tool and demonstrates the use of Python and Streamlit in developing healthcare-related solutions. It is intended to assist users with general health awareness and does not replace professional medical diagnosis or treatment.

## **Libraries Used**

| <b>Library</b> | <b>Purpose</b> |
|----------------|----------------|
|----------------|----------------|

|         |                         |
|---------|-------------------------|
| FastAPI | Backend API development |
|---------|-------------------------|

|         |                            |
|---------|----------------------------|
| Uvicorn | Running the FastAPI server |
|---------|----------------------------|

|           |                          |
|-----------|--------------------------|
| Streamlit | Frontend web application |
|-----------|--------------------------|

|          |                                                |
|----------|------------------------------------------------|
| Requests | API communication between frontend and backend |
|----------|------------------------------------------------|

|          |                                     |
|----------|-------------------------------------|
| Pydantic | Data validation and schema modeling |
|----------|-------------------------------------|

## Code Implementation

### Step 1: Install Required Libraries

First, install all required Python libraries.

```
pip install fastapi uvicorn streamlit requests pydantic
```

### Step 2: Create Backend (FastAPI)

Create a file named **main.py** and write the backend code.

- Import FastAPI and Pydantic
- Create API app
- Define input model
- Create /analyze endpoint

👉 Backend takes user symptoms as input and processes them.

### Step 3: Run Backend Server

Start the FastAPI server using Uvicorn.

```
uvicorn main:app --reload
```

- Server runs at: `http://127.0.0.1:8000`
- API endpoint becomes active

### Step 4: Create Frontend (Streamlit)

Create a file named **app.py**

- Import Streamlit and Requests
- Create UI title
- Add text input box for symptoms
- Add “Get Advice” button

### **Step 5: Connect Frontend with Backend**

When the user clicks the button:

- Streamlit sends symptoms to FastAPI
- Uses HTTP POST request
- Data is sent in JSON format

### **Step 6: Process API Response**

- Backend analyzes symptoms
- Returns medical advice
- Streamlit receives response
- Displays result on screen

### **Step 7: Symptom Analysis Logic**

The system checks input text:

- If “fever” or “cold” → returns fever/cold advice

- If “headache” → returns headache advice
- Else → asks for more details

### **Step 8: Display Output**

- Advice is shown using Streamlit
- Output appears instantly on web page

### **Step 9: Run Frontend**

Start Streamlit application:

```
streamlit run app.py
```

### **Final Output**

- User enters symptoms
- System analyzes input
- AI medical guidance is displayed
- Simple and interactive health assistant is ready

Output:

# AI Medical Assistant

Ask general health-related questions and receive AI-generated guidance.

Describe your symptoms or ask a health question:

i have cold and fever

Get Advice

## Medical Guidance

I'm here to provide helpful information. Based on your symptoms:

**1. General Information** A fever and cold are usually caused by a viral infection.

### 2. Self-Care

- Drink plenty of water
- Take enough rest